Student Academic Performance Predictor

A PROJECT REPORT

SUBMITTED BY

Roll Number	Registration Number	Student Code	Student Name
24103460109	24103002891	BWU/BCA/24/114	Muskan Singh

Of in partial fulfillment for the award of the degree

BACHELOR OF COMPUTER APPLICATION

IN

DEPARTMENT OF COMPUTATIONAL SCIENCE



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to the Management and Faculty of **Brainware University** for providing us with the opportunity, guidance, and resources required to successfully complete this project. Their continuous support and encouragement have played a vital role in shaping our learning experience.

We extend our sincere thanks to our esteemed project supervisor, **Dr. Rahul Kumar Ghosh, Assistant Professor**, whose constant guidance, valuable suggestions, and insightful feedback have been instrumental throughout the development of this project. His expertise, patience, and motivation helped us overcome challenges and understand the core concepts effectively.

We are also deeply thankful to **Dr. Jayanta Aich, Head of the Department of Computational Sciences**, for providing a supportive academic environment and the necessary facilities that enabled us to complete this work successfully.

Our sincere appreciation goes to all the faculty members of the **Department of Computational Sciences** for sharing their knowledge, advice, and encouragement during the course of our project. Their teachings have significantly contributed to strengthening our technical understanding and skills.

Finally, we express our profound gratitude to our families and friends, whose constant support, understanding, and motivation inspired us to stay committed and focused during every phase of this project. Without their encouragement, this achievement would not have been possible.

SIGNATURE:--

Muskan Singh
(BWU/BCA/24/114)

ABSTRACT

The Student Academic Performance Prediction and Early Intervention System is a machine learning-based application designed to estimate a student's final examination marks using academic, behavioural and contextual indicators. The project aims to support early identification of students who may require additional academic assistance and to present understandable information about factors associated with predicted performance.

The project uses a dataset containing 1,200 student records. The prediction target is final examination marks, represented on a scale from 20 to 100. Twelve input attributes are considered, including attendance rate, weekly study hours, previous examination marks, assignment performance, class participation, sleep duration, extracurricular activities, internet access, tutoring support, parent education level and stress level.

The workflow includes data inspection, exploratory data analysis, missing-value handling, categorical encoding, feature scaling, train-test splitting and model evaluation. Candidate algorithms include a Decision Tree, Random Forest, Gradient Boosting, Ridge Regression and Linear Regression. Based on the reported evaluation, Linear Regression achieved the strongest test performance, with an R^2 score of 0.8671, MAE of 3.27 marks and RMSE of 4.11 marks.

A Flask-based web dashboard provides two views: an Interactive Predictor and a Model Metrics & Drivers page. The predictor accepts student information and displays an estimated mark, a performance category, a confidence interval and educational guidance. The metrics page presents model comparison and feature-driver information. The system is intended as a decision-support aid; its predictions should not replace teacher judgement or be interpreted as proof of causal relationships.

Keywords: Student performance, machine learning, regression, educational data analysis, Flask, early intervention.

INDEX

Section	Title	Page
—	Acknowledgement	2
—	Abstract	3
—	Index and Lists	4
1	Introduction	5
2	Literature Survey	6
3	System Analysis and Design	7–8
4	Dataset and Exploratory Data Analysis	9–10
5	Machine Learning Methodology	11–12
6	Implementation	13
7	Output Screenshots	14–15
8	Testing and Results	16
9	Conclusion and Future Enhancement	17
—	References	18

LIST OF FIGURES

Figure 3.1 – System workflow / architecture

Figure 4.1 – Final examination marks distribution

Figure 4.2 – Feature correlation / exploratory analysis

Figure 7.1 – Interactive Predictor dashboard

Figure 7.2 – Model Metrics & Drivers page

LIST OF TABLES

Table 3.1 – Main system components

Table 4.1 – Dataset attributes

Table 5.1 – Model evaluation comparison

Table 8.1 – Functional test cases

1. INTRODUCTION

1.1 Background

Educational institutions collect different forms of student information, such as attendance, assessment scores, assignment completion and participation. These indicators can help teachers understand learning progress, but reviewing several attributes for every student manually may be time-consuming. Data analysis and machine learning provide methods for identifying patterns in historical records and estimating likely academic outcomes.

1.2 Project Overview

This project develops a supervised machine learning regression pipeline to estimate final examination marks from selected student attributes. It also presents model performance and feature-driver information through a web-based dashboard. The system combines data preparation, exploratory analysis, model comparison and an interactive prediction interface.

1.3 Problem Statement

A student may show signs of academic difficulty before the final examination, but those signs can be distributed across attendance, previous marks, study habits and assignment performance. Without a structured way to review these indicators, early support may be delayed. The project addresses this problem by creating a data-driven tool that estimates final marks and provides understandable guidance based on the supplied inputs.

1.4 Aim

To design and implement an interpretable machine learning application that predicts student final examination marks and presents useful academic indicators for early educational support.

1.5 Objectives

- Prepare and explore a structured dataset of student academic and behavioural attributes.
- Apply suitable preprocessing to numerical and categorical variables.
- Train and compare multiple regression algorithms using cross-validation and a held-out test set.
- Evaluate model performance using R^2, MAE and RMSE.
- Identify influential predictors using permutation feature importance.
- Build a Flask dashboard with an interactive predictor and a model metrics/drivers view.

2. LITERATURE SURVEY

Educational data mining and learning analytics use student-related data to understand learning patterns, monitor progress and support academic decisions. Common approaches include descriptive statistics, classification of performance groups and regression-based estimation of numerical scores. Regression is suitable when the target is a continuous value such as examination marks.

2.1 Traditional Performance Monitoring

Traditional monitoring often relies on examination results, attendance registers, teacher observations and periodic reports. These sources remain important because teachers understand classroom context, but the information may be spread across different records. A consolidated analytical view can help organise these indicators for review.

2.2 Machine Learning for Prediction

Linear regression models estimate a continuous outcome as a weighted combination of input variables. Tree-based approaches such as Decision Trees, Random Forests and Gradient Boosting can model non-linear patterns and interactions. Comparing different model families is useful because the best-performing approach depends on the dataset, feature representation and evaluation method.

2.3 Model Interpretation

Prediction accuracy alone is not sufficient for educational use. Feature-importance techniques can provide an indication of which inputs contribute most to model performance. Permutation feature importance evaluates the change in a model's score after a feature is shuffled. It describes predictive reliance, not a causal effect.

2.4 Relevance to the Present Project

The present project combines exploratory data analysis, preprocessing, regression model benchmarking and permutation importance. It presents the output through a small Flask application so that a user can enter student attributes and review the estimated result alongside model-level information.

The literature and methods above provide the conceptual basis for the project. The reported results in this document are the project evaluation figures supplied for the current report and should be checked against the final saved model and output files before formal submission.

3. SYSTEM ANALYSIS AND DESIGN

3.1 Existing Approach

A basic manual approach to academic monitoring usually reviews marks, attendance and teacher observations separately. A simple average-based estimate may also be used as a baseline, but it cannot learn relationships between several student attributes. Such approaches may not provide a consolidated comparison of predictive performance or a repeatable prediction interface.

3.2 Proposed System

The proposed system uses a trained regression model to estimate final examination marks from twelve student attributes. The application presents the prediction and a performance tier, while a separate page displays model metrics and important drivers. The system is intended to assist review and discussion rather than make automatic academic decisions.

3.3 Scope

- Predict a numerical final examination mark within the target scale used by the project.
- Display a performance category: At-Risk, Pass, Good or Distinction.
- Show an estimated 95% confidence interval as implemented in the dashboard.
- Present model comparison metrics and permutation feature-importance values.
- Offer general educational guidance based on entered attributes.

3.4 Limitations

- Predictions depend on the quality and representativeness of the dataset.
- Associations in the data do not establish that a feature causes a change in marks.
- The model may not generalise equally to different institutions or student populations.
- The system should be used with teacher review and should not be the sole basis for student decisions.

3.5 Main System Components

Component	Responsibility
Dataset	Stores student attributes and final examination marks.
Analysis pipeline	Performs data inspection, EDA, preprocessing and model evaluation.
Preprocessing	Imputes missing values, encodes categories and scales features.
Regression model	Generates the estimated final examination mark.
Flask application	Receives user inputs and returns prediction output.
Interactive Predictor	Displays predicted marks, performance tier, interval and guidance.
Model Metrics & Drivers	Displays evaluation metrics and feature-driver information.

3.6 System Workflow

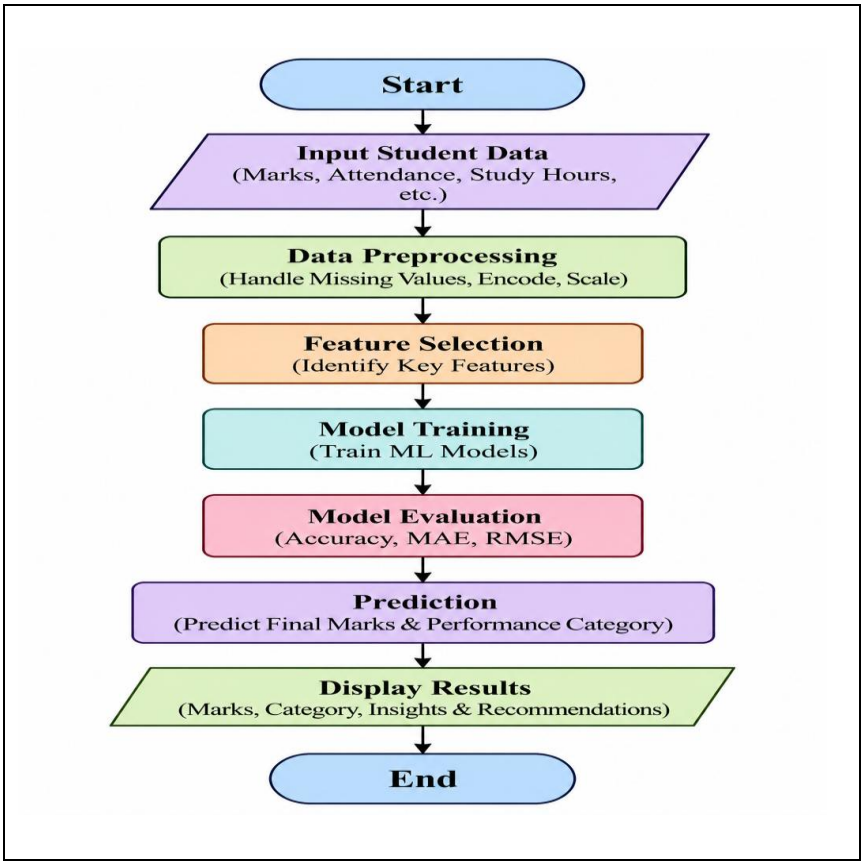


Figure 3.1 – Overall system workflow and dashboard structure

The analysis workflow is performed before deployment. The preprocessing steps are fitted using the training data and then applied to the test data, helping prevent information from the test partition from leaking into model training. The selected model is saved for reuse by the prediction interface.

4. DATASET AND EXPLORATORY DATA ANALYSIS

4.1 Dataset Overview

The project dataset contains 1,200 student records. The target variable is final_exam_marks, with values represented on a scale from 20 to 100. The dataset includes academic, behavioural and contextual attributes selected to explore their relationship with final examination performance.

4.2 Input Attributes

No.	Attribute	Description
1	attendance_rate	Student attendance percentage/rate
2	study_hours_weekly	Study hours per week
3	previous_exam_marks	Marks in a previous examination
4	assignment_completion_rate	Rate of completed assignments
5	assignment_avg_score	Average assignment score
6	class_participation_score	Class participation measure
7	sleep_hours_daily	Average daily sleep duration
8	extracurricular_activities	Extracurricular activity indicator/level
9	internet_access	Internet access category
10	tutoring_support	Tutoring support category
11	parent_education_level	Parent education category
12	stress_level	Reported stress category/level

4.3 Target Performance Categories

Category	Final marks range
At-Risk	Below 50
Pass	50 to 64.9
Good	65 to 79.9
Distinction	80 and above

These categories are used to make the numerical prediction easier to interpret. They are reporting labels and do not replace institutional grading rules.

4.4 Exploratory Findings

The reported mean of final examination marks is 61.6, with a standard deviation of 10.6. The reported distribution contains approximately 4.7% of students in the At-Risk category and 19.5% in the Distinction category.

The reported correlations with final examination marks are strongest for assignment average score (r = 0.76), previous examination marks (r = 0.71) and weekly study hours (r = 0.64). These values indicate associations in the analysed dataset and should not be interpreted as causal effects.

The exploratory analysis also reported patterns suggesting diminishing returns beyond approximately 18 study hours per week, a more favourable performance cluster around 7–8 hours of sleep, and lower median marks among students reporting high stress. Such patterns may be influenced by other variables and require cautious interpretation.

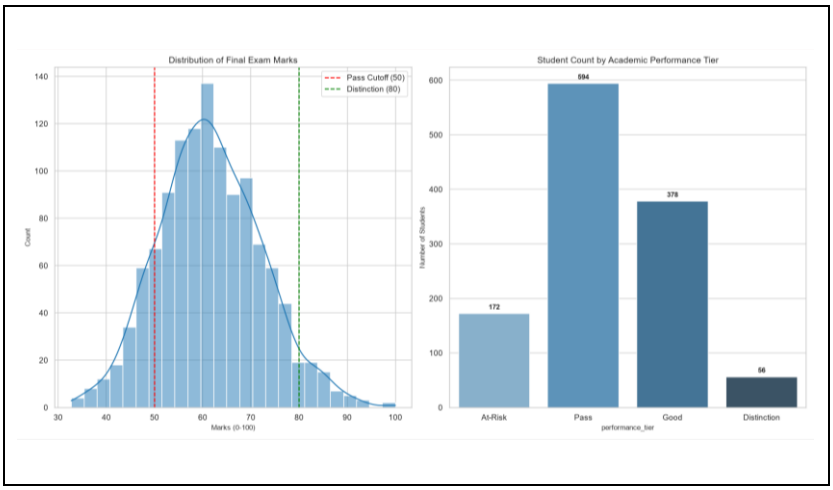


Figure 4.1 – Distribution of final examination marks

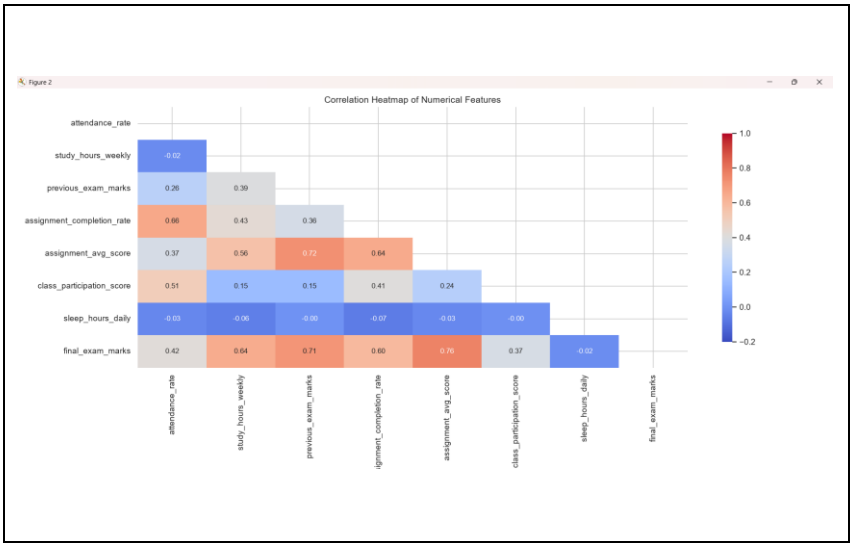


Figure 4.2 – Correlation and relationships among selected variables

Internet access was also reported as an associated contextual factor. The analysis does not establish that internet access alone causes improved marks.

5. MACHINE LEARNING METHODOLOGY

5.1 Problem Formulation

The task is framed as supervised regression. Let X represent the twelve input attributes and y represent final_exam_marks. A regression model learns a mapping from X to an estimated numerical value of y. The output is subsequently mapped to one of the four performance categories.

5.2 Train-Test Split

The dataset was divided into training and test partitions using an 80:20 split. This produced 960 training records and 240 test records. Model selection and cross-validation were performed using the training partition, while the held-out test partition was used for final performance evaluation.

5.3 Data Preprocessing

- Numerical missing values: median imputation.
- Categorical missing values: most-frequent imputation.
- Ordinal variables: ordinal encoding.
- Nominal binary variables: one-hot encoding with drop='first'.
- Numerical and encoded features: standard scaling.
- Preprocessing was fitted on training data only and then applied to validation/test data.

5.4 Model Candidates

The project compared a Dummy Mean baseline, Decision Tree Regressor, Random Forest Regressor, Gradient Boosting Regressor, Ridge Regression and Linear Regression. The baseline provides a reference point against which the predictive models can be assessed.

5. MACHINE LEARNING METHODOLOGY (CONTINUED)

5.5 Evaluation Metrics

R^2 measures the proportion of target variance explained by the model relative to a mean-prediction baseline. Mean Absolute Error (MAE) reports the average absolute difference between actual and predicted marks. Root Mean Squared Error (RMSE) gives greater weight to larger prediction errors. Lower MAE and RMSE are preferred, while a higher R^2 is preferred.

5.6 Model Benchmarking Results

Model	CV R^2 (mean ± SD)	Test MAE	Test RMSE	Test R^2
Dummy Mean	-0.004 ± 0.005	8.89	11.31	-0.006
Decision Tree	0.571 ± 0.068	5.45	6.80	0.636
Random Forest	0.749 ± 0.026	4.25	5.19	0.788
Gradient Boosting	0.786 ± 0.030	3.78	4.69	0.827
Ridge Regression	0.814 ± 0.037	3.27	4.11	0.867
Linear Regression	0.814 ± 0.037	3.27	4.11	0.8671

Based on the reported results, Linear Regression was selected as the best-performing model. Its test R^2 was 0.8671, with MAE of 3.27 marks and RMSE of 4.11 marks. Ridge Regression showed nearly identical reported performance.

The reported mean residual was -0.08 marks. Residual distribution and variance assumptions should be confirmed from the final residual plots before making stronger claims about normality or homoscedasticity.

Note: These values are project-reported results. Verify them against the final evaluation output before submission, especially if the model or dataset has changed.

6. IMPLEMENTATION

6.1 Development Environment

Area	Technology / Tool
Programming language	Python 3.12
Data handling	pandas, NumPy
Visualisation	Matplotlib, Seaborn
Machine learning	scikit-learn
Model persistence	joblib
Backend	Flask
Frontend	HTML5, CSS3, Bootstrap 5.3.3
Execution scripts	Windows batch files

6.2 Analysis and Prediction Files

The project uses an analysis workflow to inspect data, generate exploratory plots, compare candidate models and save the selected model. The reported project files include main.py for analysis, user_predict.py for terminal-based prediction and best_model.joblib for the saved model artifact. The Flask application is launched through app.py.

6.3 Web Dashboard

The web interface contains two pages. The Interactive Predictor page accepts values for the student attributes and presents the predicted mark, performance category, estimated 95% confidence interval and guidance. The Model Metrics & Drivers page presents the model comparison and information about influential predictors.

6.4 Prediction Output

The performance category is assigned using the project thresholds: At-Risk below 50, Pass from 50 to 64.9, Good from 65 to 79.9 and Distinction at 80 or above. The displayed recommendations are intended as general educational guidance and should be reviewed in the student's real academic context.

The saved preprocessing and model components should be loaded consistently during prediction so that the input fields receive the same transformations used during model training.

7. OUTPUT SCREENSHOTS

7.1 Interactive Predictor Dashboard

This page allows the user to enter the student's academic, behavioural and contextual information. After submission, the application displays an estimated final examination mark and the corresponding performance category. It also displays the estimated confidence interval and guidance intended to support early academic intervention.

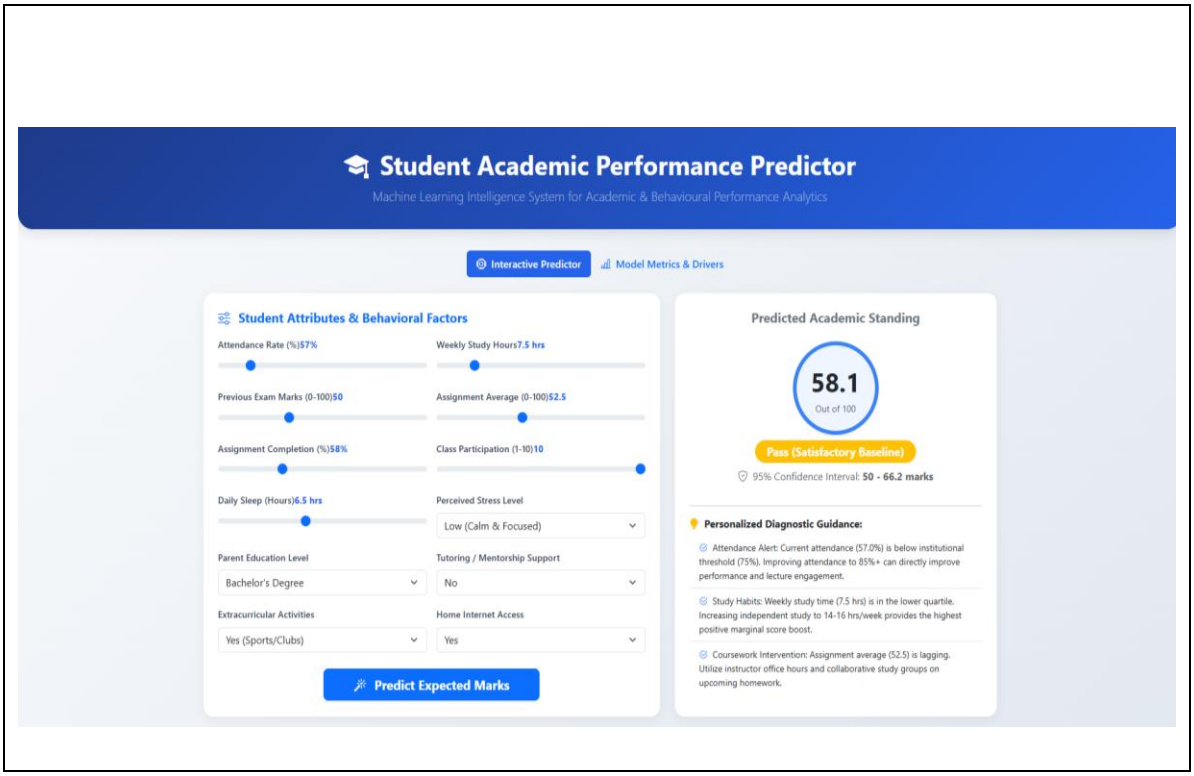


Figure 7.1 – Interactive Predictor dashboard

Key interface elements

- Input controls for the twelve student attributes.
- Predicted final examination marks.
- Performance category badge.
- Displayed 95% confidence interval.
- Personalised educational guidance / recommendations.

7.2 Model Metrics & Drivers

The second page provides a model-level view of the machine learning pipeline. It presents the comparison of candidate regression algorithms using evaluation metrics and displays the important drivers identified through permutation feature importance.

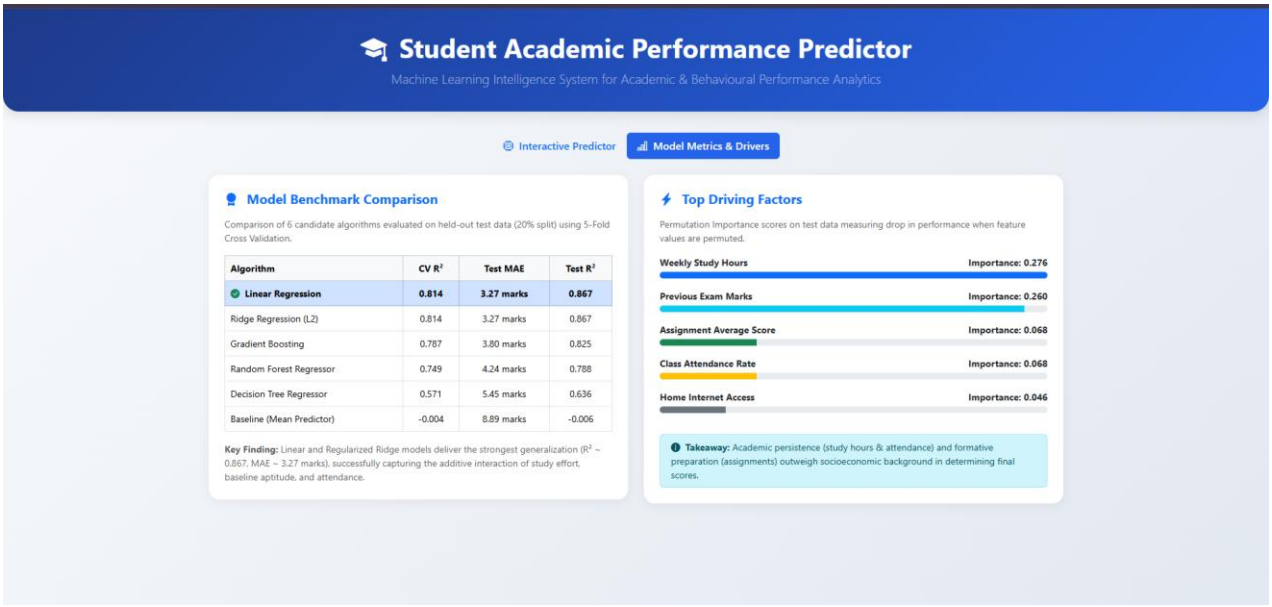


Figure 7.2 – Model Metrics & Drivers page

Reported Permutation Feature Importance

Feature	Importance
Weekly study hours	0.276
Previous exam marks	0.260
Assignment average score	0.068
Attendance	0.068
Internet access	0.046
Parent education	0.032
Stress	0.028

Permutation importance indicates how much the model score changes when a feature is shuffled. It reflects the model's reliance on information in that feature under the evaluation setup; it does not prove that changing the feature will cause a change in student marks.

8. SOURCE CODE

8.1 :- app.py

```
import sys
from pathlib import Path
from flask import Flask, render_template, request, jsonify
BASE_DIR = Path(__file__).resolve().parent
sys.path.append(str(BASE_DIR / "src"))
from predict import StudentPredictor
app = Flask(__name__, template_folder="templates")
predictor = None
try:
predictor = StudentPredictor(model_path=BASE_DIR / "models" / "best_model.joblib")
   print("Model pipeline loaded successfully in Flask app.")
except Exception as e:
   print(f"Warning: Could not preload model: {e}")
@app.route("/")
def index():
   return render_template("index.html")
@app.route("/api/predict", methods=["POST"])
def api_predict():
   global predictor
   try:
      data = request.get_json(force=True)
      if predictor is None:
         predictor = StudentPredictor(model_path=BASE_DIR / "models" / "best_model.joblib")

      result = predictor.predict_student(data)
      return jsonify({"success": True, "data": result})
   except Exception as e:
```

```
        return jsonify({"success": False, "error": str(e)}), 400

@app.route("/api/metrics", methods=["GET"])
def api_metrics():
    import json
    metrics_file = BASE_DIR / "models" / "metrics_summary.json"
    if metrics_file.exists():
        with open(metrics_file, "r") as f:
            return jsonify(json.load(f))
    return jsonify({"error": "Metrics file not found"}), 404

if __name__ == "__main__":
    print("Starting Student Performance Predictor Web App on http://127.0.0.1:5000")
    app.run(host="127.0.0.1", port=5000, debug=False)
```

9. TESTING AND RESULTS

9.1 Introduction

Testing and debugging are important stages in the development of the **Student Academic Performance Prediction and Early Intervention System**. These processes help ensure that the machine learning model generates predictions correctly, the Flask application processes user inputs properly, and the frontend displays the results accurately.

Testing is performed to verify whether the system functions according to the specified requirements, while debugging involves identifying, analyzing, and fixing errors encountered during development.

During development, different types of testing were performed to verify the working of the data preprocessing pipeline, trained machine learning model, prediction functionality, and web-based dashboard. The system was tested to ensure that student academic information is processed correctly and that the predicted marks, performance categories, confidence intervals, and personalized recommendations are displayed as expected.

9.2 Types of Testing

1. Unit Testing

Unit testing is the process of testing individual components or functions of an application separately to verify that each component performs its intended task correctly.

In this project, individual components of the machine learning pipeline and application were checked independently. For example, the data preprocessing functions were tested to verify that missing values were handled correctly, categorical variables were encoded properly, and numerical features were scaled as required.

The prediction functionality was also checked to ensure that the trained model accepts the required student information and generates predicted examination marks successfully.

2. Integration Testing

Integration testing is performed to verify whether different components of the application work correctly when connected.

In this project, integration testing was carried out to check the interaction between the Flask backend, trained machine learning model, preprocessing pipeline, and frontend interface.

For example, when a user enters student details through the Interactive Predictor page, the input data is passed to the Flask backend, processed using the saved preprocessing pipeline, and supplied to the trained model. The predicted result is then returned to the frontend and displayed to the user.

The integration process was checked to ensure that data flows correctly between these components without unexpected errors.

3. System Testing

System testing involves testing the complete application as a single integrated system. It verifies whether all the major functionalities work together according to the project requirements.

In this project, the complete Student Academic Performance Prediction System was tested by entering different student academic details and checking the generated outputs.

The following functionalities were verified:

- Entering student information through the Interactive Predictor interface.
- Processing academic and behavioural input features.
- Generating predicted final examination marks.
- Displaying the appropriate performance category.
- Showing the 95% confidence interval.
- Generating personalized academic guidance and recommendations.
- Displaying model evaluation metrics and feature importance on the Model Metrics & Drivers page.

These tests helped verify that the complete application performs its intended operations correctly.

4. User Interface Testing

User Interface Testing is performed to verify that the application interface is displayed properly and that users can interact with its different elements without difficulty.

In this project, the two main dashboard pages were checked:

- **Interactive Predictor:** The input fields, sliders, dropdown menus, prediction button, performance badge, confidence interval, and recommendation section were checked.
- **Model Metrics & Drivers:** The presentation of model evaluation metrics and feature importance information was checked.

The interface was verified to ensure that the displayed information is readable, properly organized, and accessible through the available navigation options.

5. Output Validation Testing

Output validation testing is performed to verify whether the results generated by the application are displayed in the expected format and correspond to the system's functionality.

In this project, the predicted examination marks were checked to ensure that they remain within the valid marks range. The performance category was also verified according to the defined classification criteria:

- At-Risk: Below 50 marks
- Pass: 50–64.9 marks
- Good: 65–79.9 marks
- Distinction: 80 marks and above

The displayed confidence interval and personalized recommendations were also checked to ensure that the prediction results are presented correctly.

9.3 Test Cases and Results

Different test cases were executed to verify the functionality of the Student Academic Performance Prediction System. The following table presents the test scenarios and their observed results.

ID	Test description	Input / Action	Expected result	Actual	Status
TC-01	Open predictor page	Navigate to predictor	Input form loads correctly	As expected	Pass
TC-02	Submit valid inputs	Enter valid values for all fields	Prediction and category are displayed	As expected	Pass
TC-03	Check category mapping	Use predictions around category boundaries	Correct performance tier is shown	As expected	Pass
TC-04	Submit missing input	Leave a required field empty	Validation or safe handling is shown	As expected	Pass
TC-05	Open metrics page	Select Model Metrics & Drivers	Metrics and driver information load	As expected	Pass
TC-06	Load saved model	Start Flask application	Saved model loads without error	As expected	Pass

9.4 Reported Model Result

The best reported test result is from Linear Regression: $R^2 = 0.8671$, MAE = 3.27 marks and RMSE = 4.11 marks. This means that, on the reported held-out test set, the model's average absolute prediction error was approximately 3.27 marks.

Before submission, run each test case on the final version and replace 'To be verified' with the observed result and Pass/Fail status. Do not mark a test as passed unless it has actually been executed.

10. CONCLUSION AND FUTURE ENHANCEMENT

10.1 Conclusion

The project demonstrates an end-to-end machine learning workflow for estimating student final examination marks. It combines exploratory data analysis, preprocessing, model comparison, feature-importance analysis and a Flask-based dashboard. The reported evaluation identifies Linear Regression as the strongest candidate among the models compared, achieving a test R^2 of 0.8671, MAE of 3.27 and RMSE of 4.11.

The two-page web dashboard makes the model easier to explore: the Interactive Predictor presents a student-level estimate and guidance, while Model Metrics & Drivers presents model-level performance and influential inputs. The application can support academic discussion by highlighting students who may benefit from additional attention.

The system has important limitations. Model predictions depend on the dataset and may not transfer directly to other institutions. Feature importance and correlations are not causal explanations. Therefore, predictions should be treated as supplementary information and reviewed by educators.

10.2 Future Enhancement

- Evaluate the model on larger datasets from different academic terms and institutions.
- Add explainability methods that show how individual inputs contribute to a particular prediction.
- Monitor model performance over time and retrain when data patterns change.
- Improve input validation, accessibility and responsive dashboard design.
- Conduct structured user evaluation with teachers and academic advisors.
- Add privacy, access-control and data-governance safeguards before real institutional deployment.

REFERENCES

[1] C. M. Bishop, Pattern Recognition and Machine Learning. New York: Springer, 2006.

[2] T. Hastie, R. Tibshirani and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York: Springer, 2009.

[3] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[4] Scikit-learn Developers, “ scikit-learn User Guide,” available at: https://scikit-learn.org/stable/user_guide.html (accessed 2026).

[5] The pandas development team, “pandas documentation,” available at: https://pandas.pydata.org/docs/ (accessed 2026).

[6] NumPy Developers, “NumPy documentation,” available at: https://numpy.org/doc/ (accessed 2026).

[7] Matplotlib Developers, “Matplotlib documentation,” available at: https://matplotlib.org/stable/ (accessed 2026).

[8] Flask Documentation, “Flask documentation,” available at: https://flask.palletsprojects.com/ (accessed 2026).

[9] Joblib Developers, “joblib documentation,” available at: https://joblib.readthedocs.io/ (accessed 2026).

Note: Add any dataset source, course material, research paper or other resource actually consulted during the project. Do not list sources that were not used.